
AiR Lab

코딩 세미나 3주차

2025. 8. 27.

전준

|| 목 차 ||

I . ViT 모델 탐색	1
II . 최적화 기법 적용	3
III . 하이퍼파라미터 조정	4

AiR Lab 코딩 세미나 3주차

I ViT 모델 탐색

□ ViT 모델 탐색

- 이전의 실험 결과들을 기반으로 CNN모델보다 vit 모델의 성능이 전체적으로 더 뛰어나다는 것을 확인함
- 따라서 몇 가지의 ViT 모델 중 연산량이 비슷한 세 종류의 모델들의 성능을 비교하여 최종 실험 모델 도출 (너무 연산량이 큰 모델은 실험 시간이 매우 길어지거나 memory over가 발생할 것으로 예상해, ViT모델 중 연산량이 적은 모델들로 구성)
- 모든 모델은 pin_memory = True, Cosine Annealing LR, Warmup, 244*244 resize, seed(40), AdamW, 40epoch, lr: 1e-3을 적용하였음.

모델	GFLOPS (입력 244 * 244 기준)	top-1 val/acc
ViT Small	약 4-5 G	73.18 / 65.8
DeiT Small	약 4.6 G	73.18 / 65.8
Swin Tiny	약 4.5 G	72.54 / 68.46

표1. 경량화된 ViT 계열 모델들의 연산량 및 성능 비교

- 세 모델 중 Swin_Tiny 모델이 연산량은 비슷하지만, val/acc 점수가 더 높기에 이후의 실험은 Swin_Tiny로 진행하였음.

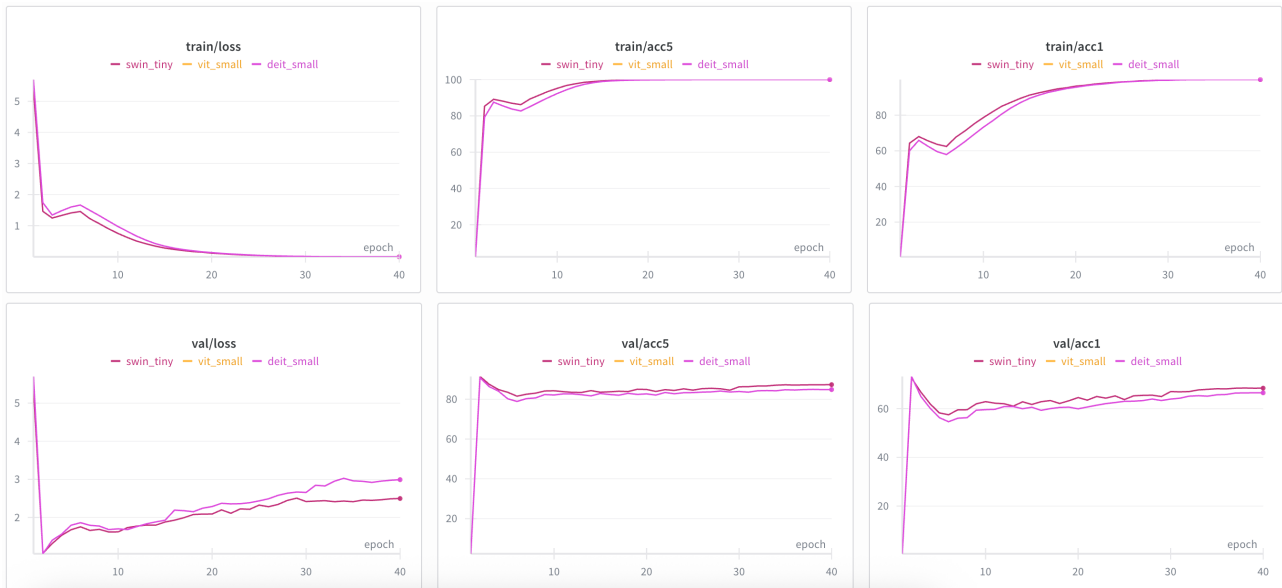


그림1. 경량화된 ViT 계열 모델들의 성능 비교 (ViT_small과 DeiT_small의 성능이 완전히 일치함.)

```

elif args.arch == 'deit_small':
    model = timm.create_model(
        'vit_small_patch16_224', pretrained=True, num_classes=200,
        drop_rate=args.drop_rate, attn_drop_rate=args.attn_drop_rate, drop_path_rate=args.drop_path_rate
    )
elif args.arch == 'vit_small':
    model = timm.create_model(
        'vit_small_patch16_224', pretrained=True, num_classes=200,
        drop_rate=args.drop_rate, attn_drop_rate=args.attn_drop_rate, drop_path_rate=args.drop_path_rate
    )
elif args.arch == 'swin_tiny':
    model = timm.create_model(
        'swin_tiny_patch4_window7_224', pretrained=True, num_classes=200,
        drop_rate=args.drop_rate, attn_drop_rate=args.attn_drop_rate, drop_path_rate=args.drop_path_rate
    )

```

그림2. 경량화 ViT 모델 코드 (main.py)

II 최적화 기법 적용

□ Swin_tiny + dropout

- 그림1의 train/acc 그래프를 보았을 때 점수가 100에 근접한 상태로 계속 학습 상태가 유지되는 과적합의 경향을 보이기에, 과적합 완화 기법인 dropout을 적용.
- drop_rate: 0.2, attn_drop_rate: 0.1, drop_path_rate: 0.1
- top-1 val/acc: 73.24 / 71.28

```
parser.add_argument('--drop_rate', type=float, default=0.2)
parser.add_argument('--attn_drop_rate', type=float, default=0.1)
parser.add_argument('--drop_path_rate', type=float, default=0.1)
elif args.arch == 'deit_small':
    model = timm.create_model(
        'vit_small_patch16_224', pretrained=True, num_classes=200,
        drop_rate=args.drop_rate, attn_drop_rate=args.attn_drop_rate, drop_path_rate=args.drop_path_rate
    )
elif args.arch == 'vit_small':
    model = timm.create_model(
        'vit_small_patch16_224', pretrained=True, num_classes=200,
        drop_rate=args.drop_rate, attn_drop_rate=args.attn_drop_rate, drop_path_rate=args.drop_path_rate
    )
elif args.arch == 'swin_tiny':
    model = timm.create_model(
        'swin_tiny_patch4_window7_224', pretrained=True, num_classes=200,
        drop_rate=args.drop_rate, attn_drop_rate=args.attn_drop_rate, drop_path_rate=args.drop_path_rate
    )
```

그림3. dropout 적용 코드 (main.py)

□ Swin_tiny + @ + albuantation

- 그림6의 train/acc 그래프를 보았을 때, 아직 과적합의 경향이 나타나기 때문에, 데이터 증강을 통한 과적합 완화를 위해 albuantation을 적용
- train_trasform:

- resize(244*244)
 - HorizontalFlip(p=0.5)
 - Rotate(limit=15, p=0.5)
 - RandomBrightnessContrast(p=0.3)
- top-1 val/acc: 70.44

```
def get_train_transform():
    return A.Compose([
        A.Resize(224, 224),
        A.HorizontalFlip(p=0.5),
        A.Rotate(limit=15, p=0.5),
        A.RandomBrightnessContrast(p=0.3),
        A.GaussNoise(var_limit=(10.0, 30.0), p=0.3),
        A.Normalize(mean=(0.485, 0.456, 0.406),
                     std=(0.229, 0.224, 0.225)),
        ToTensorV2()
    ])

def get_val_transform():
    return A.Compose([
        A.Resize(224, 224),
        A.Normalize(mean=(0.485, 0.456, 0.406),
                     std=(0.229, 0.224, 0.225)),
        ToTensorV2()
    ])
```

그림4. albuantation 적용 코드 (transforms.py)

□ Swin_tiny + @ + mixup/cutmix hybrid

- mixup과 cutmix를 확률적으로 번갈아 사용하는 mixup/cutmix hybrid를 사용하여 일반화 성능 향상을 이루어내고자 함.
- top-1 val/acc: 75.88

```
def rand_bbox(size, lam):
    W = size[2]
    H = size[3]
    cut_rat = np.sqrt(1. - lam)
    cut_w = int(W * cut_rat)
    cut_h = int(H * cut_rat)
    cx = np.random.randint(W)
    cy = np.random.randint(H)
    bbx1 = np.clip(cx - cut_w // 2, 0, W)
    bby1 = np.clip(cy - cut_h // 2, 0, H)
    bbx2 = np.clip(cx + cut_w // 2, 0, W)
    bby2 = np.clip(cy + cut_h // 2, 0, H)
    return bbx1, bby1, bbx2, bby2

def apply_mixup_cutmix(images, labels, alpha=1.0, cutmix_prob=0.5):
    r = np.random.rand()
    if alpha > 0:
        lam = np.random.beta(alpha, alpha)
    else:
        lam = 1
    batch_size = images.size(0)
    index = torch.randperm(batch_size).cuda()
    if r < cutmix_prob:
        # CutMix
        bbx1, bby1, bbx2, bby2 = rand_bbox(images.size(), lam)
        images[:, :, bbx1:bbx2, bby1:bby2] = images[index, :, bbx1:bbx2, bby1:bby2]
        lam = 1 - ((bbx2 - bbx1) * (bby2 - bby1) / (images.size(-1) * images.size(-2)))
    else:
        # Mixup
        images = lam * images + (1 - lam) * images[index, :]
    labels_a, labels_b = labels, labels[index]
    return images, labels_a, labels_b, lam
```

```

def train(model, dataloader, criterion, optimizer, epoch=9999):
    acc1_meter = AverageMeter(name='accuracy top 1')
    acc5_meter = AverageMeter(name='accuracy top 5')
    loss_meter = AverageMeter(name='loss')
    n_iters = len(dataloader)
    model.train()
    for iter_idx, (images, labels, _) in enumerate(dataloader):
        images = images.cuda()
        labels = labels.cuda()
        # CutMix & Mixup 하이브리드 적용
        images, labels_a, labels_b, lam = apply_mixup_cutmix(images, labels, alpha=1.0, cutmix_prob=0.5)
        optimizer.zero_grad()
        outputs = model(images)
        loss = lam * criterion(outputs, labels_a) + (1 - lam) * criterion(outputs, labels_b)
        loss.backward()
        optimizer.step()
        acc1, acc5 = accuracy(outputs, labels, topk=(1, 5))
        loss_meter.update(loss.item(), images.shape[0])
        acc1_meter.update(acc1[0], images.shape[0])
        acc5_meter.update(acc5[0], images.shape[0])
        print(f"[Epoch {epoch}] iter {iter_idx} / {n_iters}: \tLoss {loss_meter.val:.4f}({loss_meter.avg:.4f})")
    print("")
    print(f"Epoch {epoch} training finished")
    wandb.log({
        "epoch": epoch,
        "train/loss": loss_meter.avg,
        "train/acc1": acc1_meter.avg,
        "train/acc5": acc5_meter.avg
    }, step=epoch)

```

그림5. cutmix/mixup hybrid 적용 코드 (train.py)

model	top-1 acc
Swin_tiny	68.46
Swin_tiny + dropout	71.28
Swin_tiny + @ + albumantation	70.44
Swin_tiny + @ + mixup/cutmix hb	75.88

표2. Swin_tiny 각종 최적화 기법 적용 성능 비교

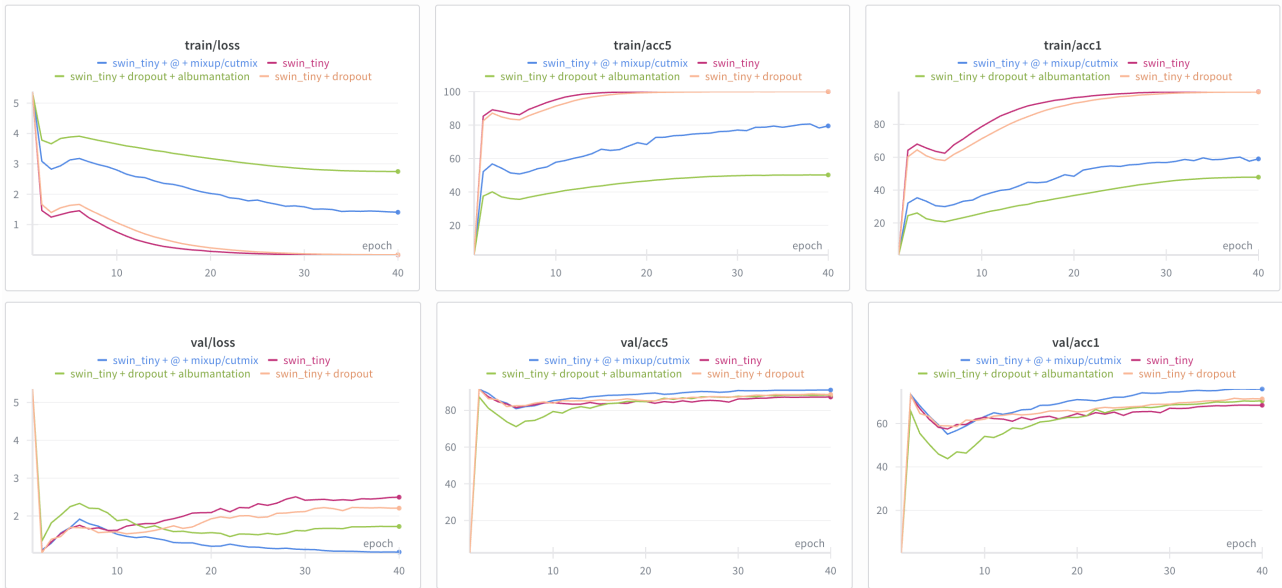


그림6. Swin_tiny 각종 최적화 기법 적용 성능 비교

III 하이퍼 파라미터 조정

□ Learning Rate $1e-3 \rightarrow 1e-5$, Warmup 제거

- 그림2의 그래프들을 보아, 학습 초기에 불안정한 모습을 보이는 것 같아 학습률을 $1e-3$ 에서 $1e-5$ 로 낮추었음.
- 초기 학습률을 매우 작은 크기인 $1e-5$ 로 낮추었기 때문에, 학습률을 기본 학습률보다 낮게 시작하여 점진적으로 복구하는 Warmup 전략이 보다 좋은 영향을 미치지 않을것이라고 판단해 Warmup 제거
- top-1 val/acc: 85.93 (20 epoch, 더 이상 의미있는 성능 향상을 기대하기 어려워 보여 일찍 학습을 종료.)

model	top-1 acc
Swin_tiny	68.46
Swin_tiny + @ + mixup/cutmix hb	75.88
Swin_tiny + @ + lr = 1e-5	85.93

표3. 기존 모델와 학습률 조정 모델 비교

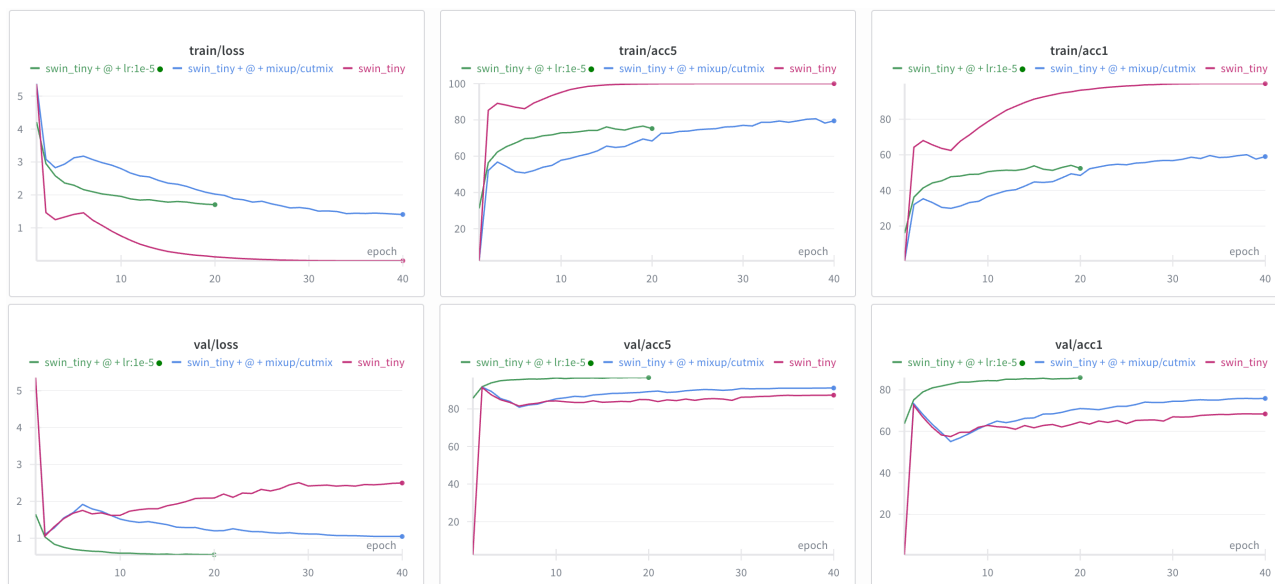


그림7. 기존 모델와 학습률 조정 모델 비교